KEPUTUSAN KEPALA BADAN TEKNOLOGI, DATA, DAN INFORMASI
KEMENTERIAN SEKRETARIAT NEGARA REPUBLIK INDONESIA

NOMOR XX TAHUN 2026

TENTANG

PEDOMAN MANAJEMEN BASIS DATA

DI KEMENTERIAN SEKRETARIAT NEGARA

1) PENDAHULUAN

a) Latar Belakang

Dokumen ini merupakan panduan wajib (*mandatory guideline*) bagi seluruh pihak yang berinteraksi dengan desain, pengembangan, dan operasional basis data relasional (SQL) di Kementerian Sekretariat Negara Republik Indonesia. Dokumen ini disusun mengacu pada kerangka kerja internasional, termasuk DAMA-DMBOK v2, ISO/IEC 25012:2008 (*Data Quality*), ISO/IEC 27001:2022 (*Information Security Management*), NIST SP 800-53, dan praktik terbaik komunitas MySQL serta PostgreSQL.

Tujuan utama panduan ini adalah menciptakan ekosistem data yang konsisten, aman, mudah dipelihara, dan siap mendukung kebutuhan analitik tingkat lanjut serta inisiatif kecerdasan buatan (AI/ML) di lingkup instansi.

b) Ruang Lingkup

Panduan ini berlaku untuk:

1) Seluruh skema basis data relasional yang berjalan di atas MySQL (versi 8.0+) dan PostgreSQL (versi 14+).
2) Semua tahap siklus hidup data: perancangan, pengembangan, pengujian, produksi, dan penonaktifan.
3) Seluruh tim yang terlibat: Software Engineer, Tim Keamanan, basis data Administrator (DBA), Data Analyst, dan Data Scientist.

c) Prinsip Dasar Desain

Seluruh desain basis data wajib berpegang pada prinsip-prinsip berikut:

i) Konsistensi

Terapkan satu aturan penamaan dan desain secara seragam di seluruh skema. Inkonsistensi, sekecil apa pun, berpotensi menimbulkan kesalahan logis (*bug*) yang sulit ditelusuri.

ii) Data sebagai Aset Strategis

Selaras dengan kerangka DAMA-DMBOK, data diperlakukan sebagai aset organisasi yang memerlukan tata kelola terstruktur dan nilai jangka panjang yang terjaga.

iii) Desain Berbasis Semantik (*Semantic-First*)

Skema basis data dirancang agar dapat dibaca dan dipahami oleh manusia maupun mesin. Gunakan nama yang jelas, berbasis domain bisnis, dan selaras dengan ontologi publik (mis. Schema.org) untuk mendukung sistem AI generatif dan text-to-SQL.

iv) Keamanan Sejak Perancangan (*Security by Design*)

Keamanan data bukan tambahan di akhir, melainkan pertimbangan utama sejak tahap perancangan awal.

v) Dapat Dipelihara (*Maintainability*)

Setiap keputusan desain harus mempertimbangkan kemudahan pemeliharaan jangka panjang, bukan hanya solusi jangka pendek.

d) Definisi

i) Data Management Body of Knowledge (DAMA-DMBOK)
DAMA-DMBOK adalah kerangka kerja manajemen data yang menjadi referensi internasional, diterbitkan oleh Data Management Association International.

ii) Data Definition Language (DDL)
Subkumpulan perintah SQL yang digunakan untuk mendefinisikan atau memodifikasi struktur objek basis data, seperti CREATE, ALTER, dan DROP.

iii) Data Manipulation Language (DML)
Subkumpulan perintah SQL yang digunakan untuk memanipulasi data, seperti SELECT, INSERT, UPDATE, dan DELETE.

iv) Orphan Record
Baris data pada tabel anak yang merujuk ke baris yang sudah tidak ada pada tabel induk, akibat tidak adanya Foreign Key constraint.

v) Personally Identifiable Information (PII)
Informasi yang dapat digunakan untuk mengidentifikasi seseorang secara langsung atau tidak langsung, seperti NIK, nama lengkap, alamat, atau nomor telepon.

vi) Point-in-Time Recovery (PITR)
Kemampuan untuk memulihkan basis data ke kondisi pada titik waktu tertentu di masa lalu, biasanya didukung oleh mekanisme WAL di PostgreSQL.

vii) Role-Based Access Control (RBAC)
Model kontrol akses di mana izin diberikan kepada peran (role), bukan langsung kepada pengguna individu.

viii) Recovery Point Objective (RPO)
Jumlah maksimum data (diukur dalam satuan waktu) yang dapat hilang saat terjadi insiden, yang masih dapat ditoleransi oleh organisasi.

ix) Recovery Time Objective (RTO)
Jumlah waktu maksimum yang dibutuhkan untuk memulihkan layanan setelah terjadi insiden, yang masih dapat ditoleransi oleh organisasi.

2) STANDAR DAN ARSITEKTUR DESAIN

a) Panduan Penamaan Universal
Konvensi penamaan berikut bersifat wajib dan berlaku secara universal untuk objek basis data tanpa terkecuali.

Aturan	Yang Dianjurkan (Do)	Yang Dilarang (Don't)
Case & Bahasa	Gunakan lowercase snake_case. → *employee_list, campus_data* Gunakan kata-kata utuh dan deskriptif.	Hindari camelCase, PascalCase. Hindari singkatan tidak standar. → *empList, CampusData, cmpsDt*

Karakter yang Diizinkan	Gunakan: a–z, 0–9, dan garis bawah (_). → *unit_kerja, total_amount*	Jangan gunakan spasi, tanda hubung (-). Jangan gunakan karakter khusus (@, #, !). → *unit-kerja, total amount*
Reserved Words	Pastikan nama tidak bentrok dengan *reserved words*. → *status_order, user_group*	Jangan gunakan kata kunci SQL sebagai nama objek. → *order, group, select, table*
Penggunaan Kutipan	Pastikan nama objek tidak memerlukan tanda kutip untuk diakses. → *unit_kerja, job_detail*	Hindari nama yang memaksa penggunaan tanda kutip ganda. → *"Job Detail", "Unit Kerja"*

b) Aturan Penamaan Objek Spesifik
Gunakan konvensi berikut untuk setiap jenis objek dalam skema *basis data*. Untuk mempermudah pemahaman skema, maka diperbolehkan untuk mengelompokkan tabel yang relevan dengan menggunakan *prefix*. Misalnya master_alamat, master_unit_kerja, riwayat_pegawai, riwayat_kehadiran.

Jenis Objek	Aturan Penamaan	Contoh
basis data	[aplikasi]_[dev/stage/prod]	*simpro_dev, simpro_stage, simpro_prod,*
Tabel	Kata benda tunggal (singular noun).	*pegawai, unit_kerja, jabatan*
Kolom	Kata benda deskriptif dalam snake_case.	*first_name, total_amount, created_at*
Kolom Boolean	Diawali dengan is_, has_, atau can_.	*is_active, has_verified_email, can_edit*
Kolom Penanggalan (Timestamp)	Diakhiri _at.	*created_at, updated_at, deleted_at*
Kolom Temporal (Date)	Diakhiri _date.	*start_date, end_date, birth_date*
Primary Key (PK)	[nama_tabel]_id.	*pegawai_id, jabatan_id, unit_kerja_id*
Foreign Key (FK)	Sama persis dengan nama PK yang direferensikan.	*pegawai_id (merujuk ke tabel pegawai)*

FK dengan Peran	[peran]_[nama_pk].	*sender_user_id, recipient_user_id*
Unique Constraint	[tabel]_[kolom(s)]_uq.	*pegawai_email_address_uq*
Check Constraint	[tabel]_[kolom]_chk.	*pegawai_umur_chk*
Index	Diawali dengan idx_ dengan format idx_[tabel]_[kolom(s)]	*idx_pegawai_last_name*
View	Diawali dengan v_.	*v_active_pegawai, v_summary_anggaran*
Stored Procedure	Diawali dengan sp_ atau proc_.	*sp_hitung_tunjangan, proc_sync_data*
Fungsi (Function)	Diawali dengan fn_.	*fn_format_nip, fn_get_jabatan*
Trigger	Diawali dengan trg_ dengan format [tabel]_[event]_trg.	*trg_pegawai_after_insert*

c) Panduan Tipe Data (MySQL & PostgreSQL)
Gunakan tipe data yang paling efisien, namun aman untuk menyimpan nilai yang dibutuhkan. Tabel berikut menyediakan padanan resmi antara MySQL dan PostgreSQL.

Entitas Bisnis	MySQL	PostgreSQL	Keterangan
Primary Key (PK)	*BIGINT UNSIGNED AUTO_INCREMENT* **(WAJIB MENGGUNAKAN UUID UNTUK MASTER DATA)**	*BIGSERIAL atau BIGINT GENERATED* **(WAJIB MENGGUNAKAN UUID UNTUK MASTER DATA)**	UUID digunakan agar primary key bersifat unik
Nilai Keuangan / Gaji	*DECIMAL(13, 4)*	*DECIMAL(13, 4) atau NUMERIC(13,4)*	Wajib untuk keakuratan data keuangan. Hindari FLOAT/DOUBLE karena rentan pembulatan oleh sistem.
Status(Opsi Terbatas)	*ENUM('PENDING','SHIPPED')*	*Tipe ENUM kustom (CREATE TYPE)*	Menegakkan integritas data di tingkat basis data.

NIK / NIP / NRP	*VARCHAR(16)*	*VARCHAR(16)*	Panjang tetap. Format string mencegah operasi matematika tidak disengaja dan mengakomodasi kombinasi huruf dan angka pada NRP
Kode Pos	*CHAR(5)*	*CHAR(5)*	Sama seperti NIK/NIP — panjang tetap dan tidak boleh dihitung secara matematis.
Alamat Email	*VARCHAR(255)*	*VARCHAR(255)*	Sesuai batas RFC 5321 untuk alamat surel.
Nama/Teks Bebas Pendek	*VARCHAR(100–500)*	*VARCHAR(100–500)*	Sesuaikan batas dengan kebutuhan bisnis.
Konten Panjang/ Narasi	*TEXT atau MEDIUMTEXT*	*TEXT*	Untuk konten tidak terstruktur yang panjang.
Waktu Pembuatan/ Modifikasi	*TIMESTAMP DEFAULT CURRENT_TIMESTAMP*	*TIMESTAMPTZ (dengan timezone)*	PostgreSQL: gunakan selalu TIMESTAMPTZ agar zona waktu tersimpan dengan benar.
Flag Boolean	*BOOLEAN atau TINYINT(1)*	*BOOLEAN*	Efisien dan ramah terhadap pemrosesan model ML.
Data Semistruktur/ Fleksibel	*JSON*	*JSONB*	JSONB di PostgreSQL lebih efisien untuk indexing dan query. Gunakan JSON di MySQL hanya bila diperlukan.
File Biner / Lampiran	*Hindari BLOB — simpan path saja*	*Hindari BYTEA — simpan path saja*	Simpan file di object storage (S3/MinIO); kolom hanya menyimpan path/URL-nya.
Nomor Handphone	*VARCHAR(20)*	*VARCHAR(20)*	Mengandung kode negara (+62)

Persentase	*DECIMAL(5,2)*	*NUMERIC(5,2)*	Wajib untuk keakuratan data keuangan. Hindari FLOAT/DOUBLE karena rentan pembulatan.
UUID (Global Identifier)	*CHAR(36)*	*UUID*	Digunakan untuk identifikasi global lintas sistem.

3) KEANDALAN DAN ANALITIK LANJUTAN

a) Jaminan Integritas Data

Integritas data adalah fondasi keandalan sistem. Seluruh perancang *basis data* wajib menerapkan aturan-aturan berikut pada tingkat skema, bukan hanya pada lapisan aplikasi.

1) Primary Keys

Setiap tabel wajib memiliki Primary Key untuk memastikan setiap baris data bersifat unik dan dapat diidentifikasi.

2) Foreign Keys

Semua relasi antar tabel wajib ditegakkan menggunakan FOREIGN KEY *constraint* untuk mencegah *orphan records* dan menjaga *referential integrity*.

3) NOT NULL

Terapkan batasan NOT NULL pada semua kolom yang secara bisnis wajib diisi. Nilai NULL hanya diperbolehkan bila secara semantik memang "belum diketahui".

4) UNIQUE Constraints

Terapkan pada kolom yang nilainya harus unik di seluruh tabel, seperti alamat email dan NIK/NIP/NRP.

5) CHECK Constraints

Gunakan CHECK untuk menegakkan aturan dan logika bisnis langsung di basis data (contoh: umur >= 18, total_harga > 0, atau status IN ('AKTIF' , 'TIDAK AKTIF')).

6) Default Values

Tetapkan nilai default yang bermakna pada kolom yang relevan (contoh: created_at DEFAULT CURRENT_TIMESTAMP, is_active DEFAULT TRUE).

b) Desain Lanjutan untuk AI dan Analitik

Perancangan basis data harus memfasilitasi kemudahan akses dan pemrosesan oleh sistem analisis, dasbor eksekutif, dan model kecerdasan buatan.

1) Tabel Metadata

   Setiap basis data yang dibuat harus menyediakan satu tabel khusus yang digunakan untuk mendeskripsikan metadata yang ada pada basis data tersebut.

2) Kolom Comment

   Setiap tabel dan kolom sensitif wajib memiliki atribut COMMENT yang menjelaskan fungsi dan konteks bisnisnya. Ini krusial bagi sistem AI generatif (text-to-SQL) agar dapat memahami data tanpa perlu dokumentasi eksternal.

3) Feature Engineering In-basis data

   Hitung fitur analitik yang sering diakses (agregasi, rasio, flag turunan) dan sediakan melalui objek VIEW atau Materialized View khusus, mengurangi beban komputasi pada sisi aplikasi.

4) Generated Columns

   Gunakan GENERATED COLUMNS untuk nilai turunan deterministik yang dihitung berdasarkan kolom lain dalam baris yang sama (contoh: kolom full_name yang digabungkan dari first_name dan last_name).

5) Data Immutability untuk Reproduksibilitas

   Terapkan pola data *immutable* (*append-only*) dengan memanfaatkan *timestamp* atau mekanisme versioning pada dataset penting yang menjadi basis pelatihan model AI, guna memastikan reproduksibilitas eksperimen.

6) Partisi Tabel

   Untuk tabel dengan volume data sangat besar (>100 juta baris), pertimbangkan partisi berdasarkan rentang waktu

(RANGE partitioning) untuk meningkatkan performa query analitik.

4) STANDARD FIELD AUDIT TRAIL

Untuk melacak riwayat (*audit trail*) setiap data dalam *basis data*, dalam pembuatan tabel wajib menambahkan 6 (enam) kolom berikut.

Nama Kolom	Tipe Data	Deskripsi
created_at	*TIMESTAMP*	Waktu data dibuat (otomatis dari sistem)
created_by	*VARCHAR(100)*	Username/ID siapa yang input data
updated_at	*TIMESTAMP*	Waktu data terakhir diubah (otomatis dari sistem)
updated_by	*VARCHAR(100)*	Username/ID siapa yang terakhir update data
deleted_at	*TIMESTAMP NULL*	Waktu data dihapus (NULL jika belum dihapus)
is_deleted	*BOOLEAN*	Flag soft delete (TRUE = sudah dihapus, FALSE = aktif)

Kolom-kolom tersebut ditambahkan hanya satu kali saat membuat tabel baru. Bukan setiap kali ada input data. Sistem akan otomatis mengisi nilai-nilai tersebut saat ada operasi INSERT, UPDATE, atau DELETE.

5) TATA KELOLA DAN OPERASIONAL

a) Strategi Backup dan Pemulihan (*Backup & Recovery*)

Strategi *backup* dirancang untuk memenuhi standar keberlangsungan layanan (*business continuity*) dan mengacu pada praktik ISO/IEC 22301.

1) Penentuan RPO dan RTO

Tentukan Recovery Point Objective (RPO) dan Recovery Time Objective (RTO) yang spesifik dan terdokumentasi untuk setiap aplikasi dan basis data.

2) Mekanisme *Backup* Berkala

   Implementasikan kombinasi Full *Backup* mingguan dan Incremental/Differential Backup harian. Untuk PostgreSQL, manfaatkan Write-Ahead Logging (WAL) untuk mengaktifkan Point-in-Time Recovery (PITR).

3) Aturan Retensi Backup (3-2-1 Rule)

   Simpan minimal 3 salinan data pada 2 media yang berbeda, dengan minimal 1 salinan berada di lokasi *off-site* atau *remote-site* yang terenkripsi.

4) Pengujian Pemulihan Berkala

   Simulasi *restore* data dilakukan minimal satu kali dalam satu tahun untuk memvalidasi integritas dan kelayakan berkas cadangan. Hasil pengujian dicatat dalam *log* resmi.

5) Enkripsi Backup

   Seluruh berkas backup wajib dienkripsi sebelum disimpan atau dikirim ke lokasi *off-site*. Gunakan standar enkripsi AES-256 atau setara.

b) Keamanan Data dan Kontrol Akses (*Security & Access Control*)

Ketentuan keamanan ini mengacu pada ISO/IEC 27001:2022 dan OWASP basis data Security Cheat Sheet.

1) Identifikasi dan Pelabelan Data Sensitif

   Setiap kolom yang menyimpan informasi pribadi yang dapat diidentifikasi (Personally Identifiable Information/PII) wajib diberikan label eksplisit melalui atribut COMMENT dengan tag [PII]. Hal ini memungkinkan sistem audit otomatis dan tools AI untuk mengenali dan membatasi eksposur data sensitif.

2) Penerapan *Least Privilege*

   Dalam penerapan *Role-Based Access Control* (RBAC) dilarang menggunakan kredensial *superuser/root* pada level aplikasi. Setiap developer akan diberikan akun user dengan

hak akses sesuai dengan perannya. Adapun *role* spesifik dibuat dan digunakan sesuai kebutuhan sebagai berikut:

a) Read Only

Untuk keperluan dasbor, Business Intelligence (BI), dan pelaporan.

b) Read Write

Untuk layanan aplikasi operasional (CRUD). Dapat diberikan kepada user developer dengan batasan waktu sesuai dengan *environment* basis data yakni satu bulan untuk Production.

c) Administrator/Superadmin

Hanya untuk eksekusi DDL (Data Definition Language) yang diotorisasi dan user management.

3) Audit Logging

Aktifkan pencatatan log keamanan (pgaudit di PostgreSQL, general_log/audit_log di MySQL) untuk melacak seluruh perubahan Data Definition Language (DDL) dan akses terhadap tabel yang mengandung informasi sensitif atau rahasia negara.

4) Data Masking / Anonymization

Seluruh data yang direplikasi dari lingkungan Production ke lingkungan Development atau Staging wajib melewati proses anonimisasi (masking) terlebih dahulu untuk melindungi informasi sensitif.

5) Enkripsi Menyeluruh (Encryption at Rest dan In-Transit)

Terapkan enkripsi untuk data yang tersimpan (at-rest) menggunakan AES-256 atau setara. Wajibkan penggunaan SSL/TLS versi 1.2 atau lebih tinggi untuk seluruh koneksi transmisi data (in-transit).

6) Pencegahan Eksploitasi Kerentanan basis data

Seluruh query yang dieksekusi dari lapisan aplikasi wajib menggunakan *parameterized queries* atau *prepared statements*. Dilarang membangun query dengan *string concatenation*.

7) Hak akses basis data harus ditinjau secara berkala, yaitu setiap jangka waktu tiga bulan atau ketika ada perubahan, untuk memastikan bahwa setiap pengguna hanya memiliki izin sesuai dengan fungsi pekerjaan mereka saat ini dan mencegah akumulasi hak akses yang tidak perlu.

c) Manajemen Lingkungan dan Migrasi Skema

Pemisahan lingkungan dan pengelolaan perubahan skema yang terdisiplin adalah kunci stabilitas sistem produksi.

1) Isolasi Lingkungan

Wajib ada pemisahan mutlak antara lingkungan Development (Dev), Staging, dan Production (Prod). Data produksi tidak boleh diakses langsung dari lingkungan pengembangan.

2) Version Control untuk Skema basis data

Dilarang keras mengeksekusi perubahan skema (DDL) secara manual langsung di lingkungan Production. Seluruh perubahan skema wajib:

a) Ditulis sebagai skrip migrasi SQL bernomor urut (*versioned migration scripts*).
b) Disimpan di repositori Git bersama kode sumber aplikasi.
c) Dieksekusi secara otomatis melalui alat migrasi.
d) Dapat di-rollback bila terjadi kegagalan (setiap skrip migrasi wajib memiliki skrip rollback yang terdefinisi).

3) Proses Review dan Approval

Setiap perubahan skema pada lingkungan Production wajib melalui proses peer review oleh minimal satu DBA lain dan mendapat persetujuan (*approval*) dari penanggung jawab teknis sebelum dieksekusi.

d) Pemeliharaan dan Optimasi Kinerja

Pemeliharaan yang proaktif dapat mencegah penurunan performa yang sering kali baru disadari saat sistem sudah mengalami gangguan.

1) Perawatan Terjadwal (*Scheduled Maintenance*)

   Konfigurasikan dan pantau proses pemeliharaan bawaan seperti Autovacuum di PostgreSQL untuk mencegah *table bloat* dan memperbarui statistik planner atau ANALYZE dan OPTIMIZE TABLE di MySQL secara terjadwal.

2) Manajemen Indeks

   Evaluasi utilitas indeks secara berkala menggunakan query statistik penggunaan. Hapus indeks yang tidak pernah digunakan (*unused indexes*) untuk mengurangi beban operasi tulis (INSERT/UPDATE/DELETE) dan konsumsi *storage*.

3) Connection Pooling

   Wajib mengimplementasikan pengelola koneksi (PgBouncer untuk PostgreSQL, ProxySQL untuk MySQL) pada layanan dengan lalu lintas konkurensi tinggi, untuk mencegah kelebihan beban koneksi pada server basis data.

4) Query Performance Monitoring

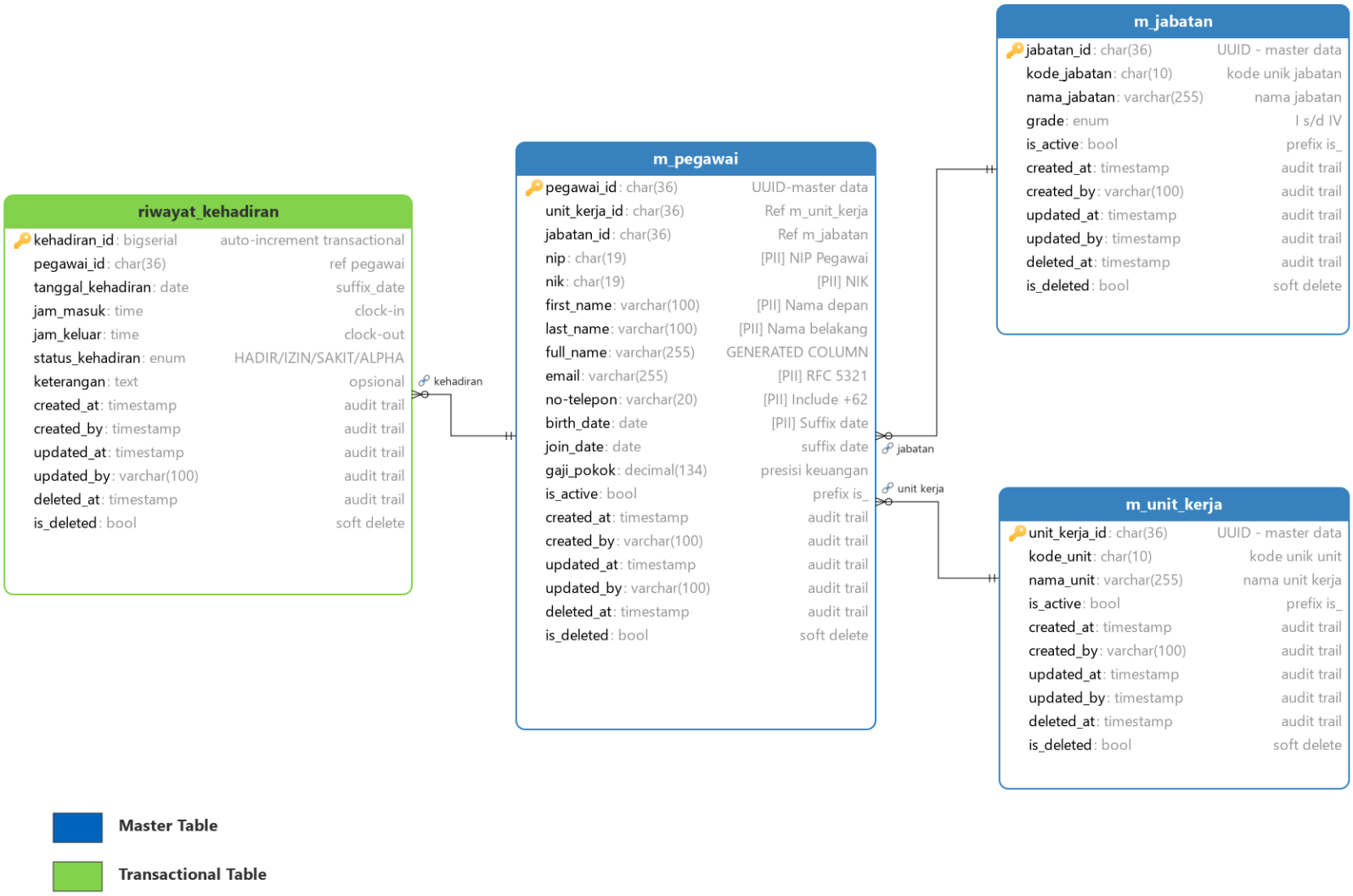
   Mengaktifkan dan memantau log slow query (slow_query_log di MySQL, log_min_duration_statement di PostgreSQL). Gunakan alat EXPLAIN/EXPLAIN ANALYZE untuk mendiagnosis dan mengoptimalkan query yang berperforma buruk.

5) Monitoring dan Alerting

   Mengintegrasikan *basis data* dengan sistem monitoring (Prometheus + Grafana, Datadog, atau setara) untuk memantau metrik utama yaitu penggunaan CPU, memori, disk I/O, jumlah koneksi aktif, dan replication lag.

6) ENTITY RELATION DIAGRAM (ERD), GLOSARIUM, DAN REFERENSI

a) Contoh Entity Relation Diagram (ERD)



b) Referensi Standar dan Sumber

No.	Standar / Dokumen	Keterangan
1	**DAMA-DMBOK v2 (2017)**	Kerangka kerja manajemen data yang menjadi landasan konsep tata kelola dalam dokumen ini.
2	**ISO/IEC 25012:2008**	Standar kualitas data — mendefinisikan karakteristik kualitas data yang perlu dikelola.
3	**ISO/IEC 27001:2022**	Standar Sistem Manajemen Keamanan Informasi (ISMS) — menjadi acuan Pasal 8 dokumen ini.
4	**ISO/IEC 22301:2019**	Standar Business Continuity Management — menjadi acuan strategi backup dan pemulihan di Pasal 7.
5	**NIST SP 800-53 Rev. 5**	Panduan kontrol keamanan dan privasi dari National Institute of Standards and Technology (NIST).
6	**OWASP basis data Security Cheat Sheet**	Praktik terbaik keamanan basis data dari Open Web Application Security Project (OWASP).
7	**MySQL 8.0 Reference Manual**	Dokumentasi resmi MySQL dari Oracle Corporation.
8	**PostgreSQL 16 Documentation**	Dokumentasi resmi PostgreSQL dari The PostgreSQL Global Development Group.
9	**Schema.org**	Ontologi data terstruktur publik — menjadi referensi konvensi penamaan berbasis semantik.

UNTUK KEPERLUAN INTERNAL

*Dokumen ini adalah milik Kementerian Sekretariat Negara Republik Indonesia. Dilarang menyebarluaskan, mereproduksi, atau menggunakan isi dokumen ini di luar lingkungan instansi tanpa izin tertulis dari pemilik dokumen.***DOKUMEN RESMI — HANYA UNTUK PENGGUNAAN INTERNAL**

Struktur Tabel Metadata

Tabel 1. metadata_data

Satu baris untuk setiap data. Relasi: metadata_data (1) ke metadata_atribut (N).

Nama Kolom	Tipe Data	Deskripsi
id_data (PK)	*INT*	Nomor unik sebagai identitas metadata data
nama_data	*VARCHAR(255)*	Nama data yang dikelola oleh unit kerja/perangkat daerah
id_probis	*INT*	ID proses bisnis terkait
probis_terkait	*VARCHAR(255)*	Nama proses bisnis terkait
uraian_data	*TEXT*	Penjelasan atau keterangan dari data, terutama mengenai ciri khas data tersebut
tujuan_data	*TEXT*	Penjelasan tujuan dari pemanfaatan data
kode_model_referensi	*VARCHAR(50)*	Pilihan kode model referensi data yang sesuai dengan data yang dipilih
sifat_data	*VARCHAR(50)*	Klasifikasi data berdasarkan kerahasiaan konten data
jenis_data	*VARCHAR(50)*	Klasifikasi data berdasarkan bentuk datanya
validitas_data	*VARCHAR(100)*	Tanggal/periode berlakunya data
penghasil_data	*VARCHAR(255)*	Penghasil/produsen data
penanggung_jawab	*VARCHAR(255)*	Penanggung jawab data/wali data
informasi_terkait_input	*TEXT*	Keterhubungan terhadap informasi yang dibutuhkan
informasi_terkait_output	*TEXT*	Keterhubungan terhadap informasi yang dihasilkan
interoperabilitas	*VARCHAR(10)*	Penjelasan apakah terdapat interoperabilitas data dengan data yang lain (Ya/Tidak)
id_metadata_terkait	*VARCHAR(50)*	Mengacu kepada metadata SPBE terkait
created_at	*TIMESTAMP*	Waktu data dibuat (otomatis dari sistem)
created_by	*VARCHAR(100)*	Username/ID siapa yang input data
updated_at	*TIMESTAMP*	Waktu data terakhir diubah (otomatis dari sistem)
updated_by	*VARCHAR(100)*	Username/ID siapa yang terakhir update data
deleted_at	*TIMESTAMP NULL*	Waktu data dihapus (NULL jika belum dihapus)
is_deleted	*BOOLEAN*	Flag soft delete (TRUE = sudah dihapus, FALSE = aktif)

Tabel 2. metadata_atribut

Banyak baris untuk setiap data, karena atribut data dapat terdiri lebih dari satu.

Nama Kolom	Tipe Data	Deskripsi
id_atribut (PK)	*INT*	Nomor unik atribut (auto increment)
id_data (FK)	*INT*	Mengacu ke metadata_data.id_data
atribut_data	*VARCHAR(100)*	Nama atribut data
mark	*VARCHAR(20)*	Penanda kunci atribut (PK/FK)
jenis_data	*VARCHAR(50)*	Tipe data atribut (teks, tanggal, angka, video, gambar, dll.)
panjang_maks	*INT*	Panjang karakter maksimal
mandatory	*BOOLEAN*	Wajib diisi atau tidak (TRUE = wajib, FALSE = tidak wajib)
aturan_bisnis	*TEXT*	Aturan pengisian/validasi atribut
created_at	*TIMESTAMP*	Waktu data dibuat (otomatis dari sistem)
created_by	*VARCHAR(100)*	Username/ID siapa yang input data
updated_at	*TIMESTAMP*	Waktu data terakhir diubah (otomatis dari sistem)
updated_by	*VARCHAR(100)*	Username/ID siapa yang terakhir update data
deleted_at	*TIMESTAMP NULL*	Waktu data dihapus (NULL jika belum dihapus)
is_deleted	*BOOLEAN*	Flag soft delete (TRUE = sudah dihapus, FALSE = aktif)